

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	N.KISHAN KUMAR	Reg. No.:	192424404
Section / Batch:	B.Tech AI & DS	Date:	08-08-2026

Code of Conduct

I N.kishan Kumar (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
 - Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: CFG Rule Validation and Parse Tree Construction

The given CFG is $S \rightarrow NP VP$, $NP \rightarrow Det N$, and $VP \rightarrow V NP$, with determiners the/a, nouns student/teacher/book, and verbs reads/likes.

Aim

To design a Python program to validate whether a given English sentence follows the specified Context-Free Grammar (CFG) and construct its parse tree.

Concept

A Context-Free Grammar (CFG) defines the syntactic structure of a language using production rules.

Here, a valid sentence must follow:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$Det \rightarrow the | a$

$N \rightarrow student | teacher | book$

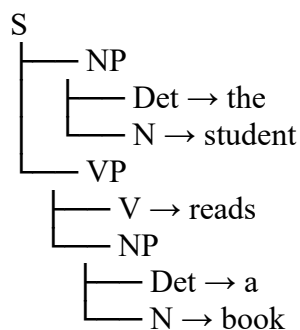
$V \rightarrow reads | likes$

Therefore, the sentence structure is:

Determiner + Noun + Verb + Determiner + Noun

Example:

the student reads a book



Algorithm

1. Accept a sentence from the user.
2. Split the sentence into words.
3. Check whether it contains exactly five words.
4. Check each word according to the CFG rules.
5. If all rules match, construct the parse tree.
6. Otherwise, return Invalid Sentence.

Python Program

```
def parse_cfg(sentence):
    w = sentence.split()
    if len(w) != 5:
        return "Invalid Sentence"
    det = ["the", "a"]
    noun = ["student", "teacher", "book"]
    verb = ["reads", "likes"]
    if (w[0] in det and w[1] in noun and
        w[2] in verb and w[3] in det and
        w[4] in noun):
```

```

    return f""S
    └─ NP
        ├── Det → {w[0]}
        └─ N → {w[1]}
    └─ VP
        ├── V → {w[2]}
        └─ NP
            ├── Det → {w[3]}
            └─ N → {w[4]} ""

```

```

    return "Invalid Sentence"
tests = [
    "the student reads a book",
    "a teacher likes the book",
    "student reads book",
    "the book likes a teacher",
    "reads the student book"
]
for sentence in tests:
    print("\nSentence:", sentence)
    print(parse_cfg(sentence))

```

Expected Output

Sentence: the student reads a book
Valid Parse Tree

Sentence: a teacher likes the book
Valid Parse Tree

Sentence: student reads book
Invalid Sentence

Sentence: the book likes a teacher
Valid Parse Tree

Sentence: reads the student book
Invalid Sentence

These validity results match the test cases specified in the assessment.

Result

Thus, the Python program successfully validates sentences using CFG production rules and generates a parse tree for grammatically valid sentences.

Experiment 2: Agreement and Feature Structure Checking

The experiment requires checking subject–verb agreement using **Number and Person attributes**. The supplied rules classify he, she, and it as singular third person, while they is plural.

Aim

To implement a Python program that verifies subject–verb agreement using feature structures.

Concept

Subject–verb agreement means that the verb must agree with its subject in grammatical features such as number.

For example:

he runs → Correct

he run → Incorrect

they run → Correct

they runs → Incorrect

Feature structures:

he → singular, third person

she → singular, third person

it → singular, third person

they → plural

runs, writes → singular verbs

run, write → plural verbs

Algorithm

1. Accept the subject and verb.
2. Store subject features in a dictionary.
3. Store verb features in another dictionary.
4. Obtain the number feature of the subject.
5. Obtain the number feature of the verb.
6. Compare the two features.
7. Return True when they agree; otherwise return False.

Python Program

```
def check_agreement(subject, verb):
```

```
    subjects = {
        "he": "singular",
        "she": "singular",
        "it": "singular",
        "they": "plural"
    }
```

```
    verbs = {
        "runs": "singular",
        "writes": "singular",
        "run": "plural",
        "write": "plural"
    }
```

```
    if subject not in subjects or verb not in verbs:
        return False
```

```
    return subjects[subject] == verbs[verb]
```

```
tests = [
    ("he", "runs"),
    ("he", "run"),
    ("they", "run"),
```

```

    ("they", "writes"),
    ("she", "writes")
]

```

```

for subject, verb in tests:
    print(subject, verb, "→",
          check_agreement(subject, verb))

```

Output

he runs → True

he run → False

they run → True

they writes → False

she writes → True

These are the expected agreement results given in the assignment.

Result

Thus, the program successfully verifies subject–verb agreement by comparing grammatical feature structures of subjects and verbs.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

The supplied PCFG assigns probabilities John = 0.6, Mary = 0.4, and runs/walks = 0.5, while $S \rightarrow NP VP$ has probability 1.0.

Aim

To implement a Python program to parse a sentence and calculate its probability using a Probabilistic Context-Free Grammar (PCFG).

Concept

A **Probabilistic Context-Free Grammar (PCFG)** assigns a probability to each grammar production.

Given grammar:

$S \rightarrow NP VP$ [1.0]

$NP \rightarrow \text{John}$ [0.6]

$NP \rightarrow \text{Mary}$ [0.4]

$VP \rightarrow \text{runs}$ [0.5]

$VP \rightarrow \text{walks}$ [0.5]

The sentence probability is calculated as:

$P(\text{Sentence})$

$= P(S \rightarrow NP VP) \times P(NP) \times P(VP)$

For example:

John runs

$P = 1.0 \times 0.6 \times 0.5$

$P = 0.30$

Algorithm

1. Accept a two-word sentence.
2. Split it into noun and verb.
3. Check whether the noun exists in the NP grammar.
4. Check whether the verb exists in the VP grammar.

5. Retrieve their corresponding probabilities.
6. Multiply the grammar probabilities.
7. Return 0.00 when the sentence cannot be generated.

Python Program

```
def sentence_probability(sentence):
    words = sentence.split()
    if len(words) != 2:
        return 0.0
    np_prob = {
        "John": 0.6,
        "Mary": 0.4
    }
    vp_prob = {
        "runs": 0.5,
        "walks": 0.5
    }
    noun, verb = words
    if noun not in np_prob or verb not in vp_prob:
        return 0.0
    return 1.0 * np_prob[noun] * vp_prob[verb]
tests = [
    "John runs",
    "John walks",
    "Mary runs",
    "Mary walks",
    "Peter runs"
]
for sentence in tests:
    print(sentence, "→",
          f"{sentence_probability(sentence):.2f}")
```

Output

John runs → 0.30
John walks → 0.30
Mary runs → 0.20
Mary walks → 0.20
Peter runs → 0.00

These probabilities correspond exactly to the test cases in the provided assessment.

Sample Calculations

1. John runs

$P(\text{John runs})$
 $= P(S \rightarrow NP VP) \times P(NP \rightarrow \text{John}) \times P(VP \rightarrow \text{runs})$
 $= 1.0 \times 0.6 \times 0.5$
 $= 0.30$

2. Mary walks

$P(\text{Mary walks})$
 $= 1.0 \times 0.4 \times 0.5$
 $= 0.20$

3. Peter runs

Peter is not present in the NP production rules.

Therefore,
 $P(\text{Peter runs}) = 0.00$

Result

Thus, the PCFG program successfully parses valid sentences and calculates their probabilities using probabilistic production rules. Invalid sentences are assigned a probability of 0.00.

Overall Conclusion

The three experiments demonstrate important concepts of Natural Language Processing and syntactic analysis. Experiment 1 uses CFG production rules to validate sentence structure and construct parse trees. Experiment 2 applies feature structures to check subject–verb agreement. Experiment 3 extends CFG with probabilities using PCFG to calculate the likelihood of valid sentences. These experiments demonstrate rule-based, feature-based, and probabilistic approaches to syntactic analysis.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
-------------------	--------------------	------------------

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____